Dart OOP Challenging Tasks for Students

1. Bank Account System with Inheritance and Encapsulation

Objective: Create a bank system with multiple types of accounts and perform various banking operations.

Instructions:

- Create a base class `Account` with fields for `accountNumber`, `balance`, and `accountHolderName`.
- Create methods in `Account` for `deposit()`, `withdraw()`, and `checkBalance()`.
- Extend `Account` to create two types of accounts: `SavingsAccount` and `CurrentAccount`.
- `SavingsAccount` should have an interest rate field and a method to apply interest to the balance.
- `CurrentAccount` should have an overdraft limit and should not allow withdrawal beyond that limit.
- Use encapsulation to ensure the balance is private and can only be accessed via the methods.

2. Library System with Multiple Types of Books

Objective: Create a library system where students can borrow and return books.

Instructions:

- Create an abstract class `Book` with methods `borrow()` and `returnBook()`.
- Create subclasses `EBook` and `PrintedBook`.
- `EBook` should have a method for downloading the book, and it should also have a `fileSize` property.
- `PrintedBook` should have a method for checking if the book is available at the library.
- Implement polymorphism: the `borrow()` method should behave differently for `EBook` and `PrintedBook`.
- Create a `Library` class that can keep track of books, check out books, and allow students to return

them.

- Use encapsulation to manage the availability of the books (private property).

3. Shape Area Calculation with Multiple Inheritance

Objective: Calculate the area of various shapes and use mixins to add additional functionality.

Instructions:

- Create a class `Shape` with a method `calculateArea()`.
- Create subclasses `Rectangle`, `Circle`, and `Triangle`.
- `Rectangle` should have `length` and `width`.
- `Circle` should have `radius`.
- `Triangle` should have `base` and `height`.
- Create mixins for specific calculations:
  - `PerimeterMixin` for calculating the perimeter of a shape.
  - `VolumeMixin` for 3D shapes.
- Use mixins in relevant shape classes.

4. School Management System with Polymorphism and Interfaces

Objective: Design a school management system where students, teachers, and staff have different roles, but share common functionality.

Instructions:

- Create an interface `Person` with methods `displayDetails()` and `getRole()`.
- Implement the `Person` interface in classes `Student`, `Teacher`, and `Staff`.
- Each class should have specific properties like `name`, `id`, and `subject` for `Teacher`, `grade` for `Student`, and `department` for `Staff`.
- The `displayDetails()` method should print details specific to the type of person.
- The `getRole()` method should return the role as a string.

- Use polymorphism to call `displayDetails()` on a list of mixed `Person` types.

5. Game Characters with Mixins and Abstraction

Objective: Design a game system where characters can have different abilities (e.g., attack, defend, heal) and use mixins to share these abilities.

Instructions:

- Create an abstract class `Character` with methods `attack()`, `defend()`, and `heal()`.
- Create mixins `Attacker`, `Defender`, and `Healer`, each having one of the corresponding methods.
- Create classes `Warrior`, `Mage`, and `Archer` that extend `Character` and mix in the relevant abilities.
- Demonstrate polymorphism by creating a list of mixed `Character` objects and performing actions like `attack()`, `defend()`, and `heal()`.

6. E-Commerce System with Polymorphism and Interfaces

Objective: Design an e-commerce system where products can have different types (e.g., `PhysicalProduct`, `DigitalProduct`), and customers can place orders.

Instructions:

- Create an interface `Product` with methods `getPrice()` and `getDescription()`.
- Create classes `PhysicalProduct` and `DigitalProduct` that implement `Product`.
- `PhysicalProduct` should have properties like `weight` and `shippingCost`.
- `DigitalProduct` should have properties like `fileSize` and `downloadLink`.
- Create a `ShoppingCart` class that can hold a list of `Product` objects and calculate the total price.
- Create a `Customer` class that can place orders.
- Demonstrate polymorphism by treating all `Product` objects uniformly in the shopping cart.

7. Dynamic Configuration System with Mixins and Inheritance

Objective: Design a system to manage dynamic configurations that can be updated at runtime.

Instructions:

- Create an abstract class `Configurable` with methods `applyConfig()` and `resetConfig()`.
- Create mixins `NetworkConfig`, `DatabaseConfig`, and `UIConfig`, each of which applies specific configurations.
- Create classes `AppConfig`, `ServerConfig`, and `UserConfig` that extend `Configurable` and mix in the relevant configurations.
- Demonstrate polymorphism by treating all configurations uniformly and applying them at runtime.

8. Shopping Cart with Multiple Discount Strategies (Strategy Pattern)

Objective: Implement a shopping cart with different discount strategies.

Instructions:

- Create an interface `DiscountStrategy` with a method `applyDiscount()`.
- Create classes for different discount strategies: `PercentageDiscount`, `FlatDiscount`, and `FreeShipping`.
- Create a `ShoppingCart` class that can use any `DiscountStrategy` to apply discounts.
- Demonstrate the use of strategy pattern by applying different discount strategies based on the user's choice.

9. Vehicle Management System with Inheritance and Polymorphism

Objective: Design a vehicle management system with different types of vehicles that share common behavior.

Instructions:

- Create an abstract class `Vehicle` with a method `start()`.

- Create subclasses `Car`, `Bike`, and `Truck` that inherit from `Vehicle`.
- `Car` should have additional properties like `fuelType`.
- `Bike` should have properties like `engineType`.
- `Truck` should have properties like `loadCapacity`.
- Override `start()` in each subclass to implement specific behavior for starting the vehicle.
- Use polymorphism to create a list of mixed `Vehicle` objects and call `start()` on them.

10. Real-Time Notifications System

Objective: Design a notification system where different types of notifications (e.g., `EmailNotification`, `SMSNotification`) can be sent based on user preferences.

Instructions:

- Create an abstract class `Notification` with methods `send()` and `setRecipient()`.
- Create subclasses `EmailNotification` and `SMSNotification` that override `send()` to send notifications via email and SMS, respectively.
- Create a `User` class that stores notification preferences and sends notifications based on their preference.
- Demonstrate polymorphism by creating a list of `Notification` objects and calling the `send()` method.